

모두를 위한 딥러닝 (이미지처리편)

경북대학교 IT교육센터

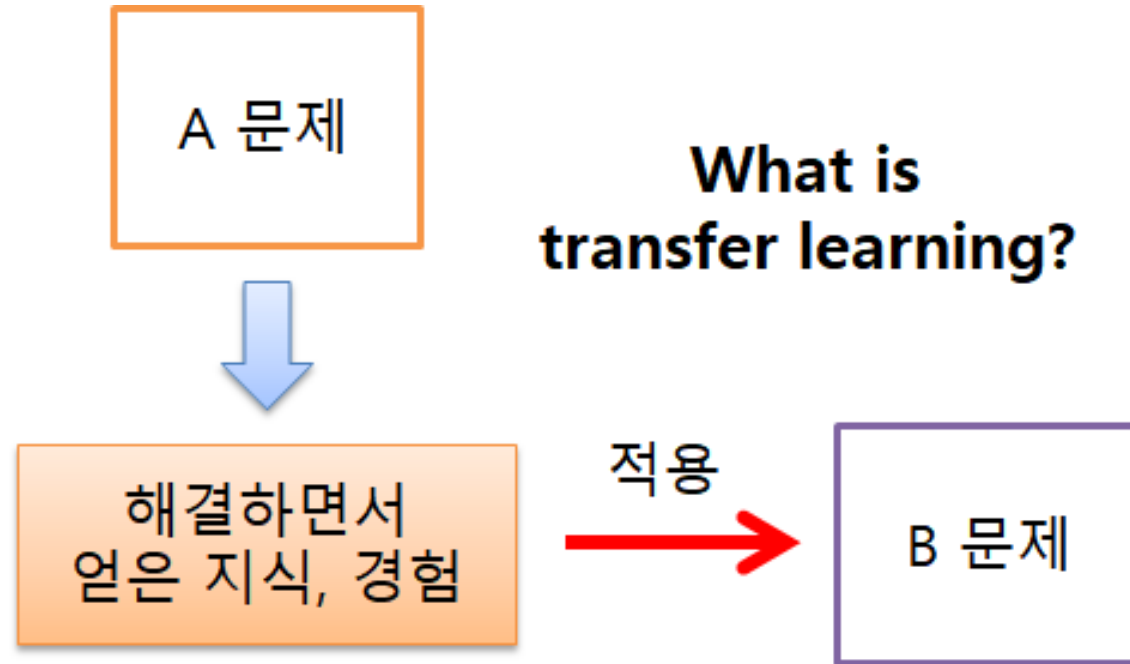
허 찬

차시(모듈)	차시별 주제	학습내용
1	딥러닝을 이용한 이미지 처리	이미지 처리의 이해 다양한 이미지처리의 응용 분야에 대한 이해
2	특징 추출의 이해	이미지 데이터 전처리 기법의 이해 이미지 특징 추출의 이해 및 실습 딥러닝을 이용한 학습 및 추론 과정의 이해
3	Convolution Neural Network	합성곱 신경망의 이해
4	CNN 모델의 이해	다양한 CNN 모델의 이해 딥러닝의 여러가지 요소 이해
5	CNN을 이용한 분류 모델 실습	CNN을 이용한 이미지 분류 모델 실습
6	전이 학습 모델의 이해	전이 학습의 이해 전이학습을 이용한 이미지 처리 모델 실습
7	객체 인식 모델의 이해	다양한 객체 인식 모델의 이해 Semantic segmentation (U-Net)의 이해
8	Transformer 기반 이미지 처리 모델 1	Attention과 Transformer의 이해
9	Transformer 기반 이미지 처리 모델 2	Transformer 기반 이미지 처리 모델 (ViT)의 이해
10	이미지 생성 모델의 이해	최신 이미지 생성 모델의 이해 오토인코더의 이해 및 실습
11	최신 이미지 처리 기술의 이해	최신 컴퓨터비전 모델의 이해 비디오 처리의 이해

Transfer learning (전이학습)

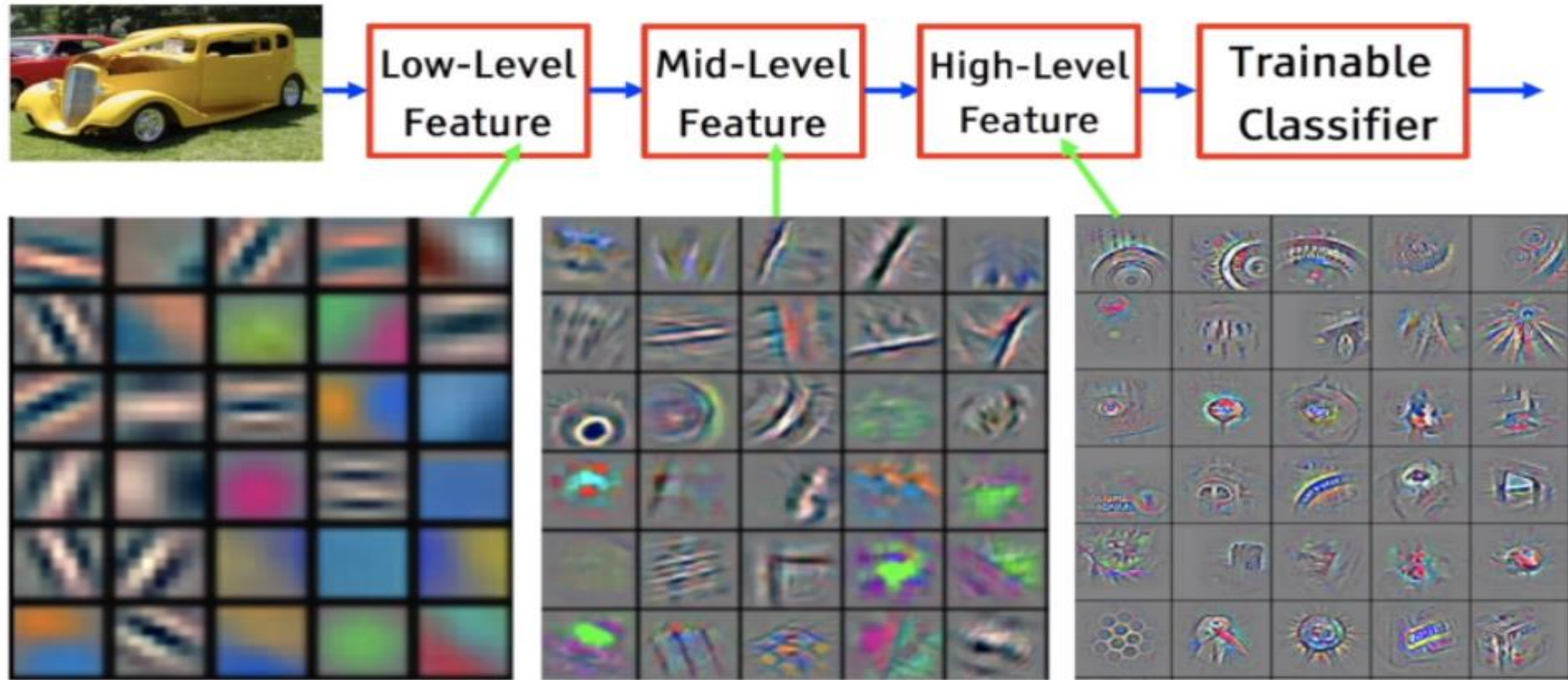
- 한 분야의 문제를 해결하기 위해서 얻은 지식과 정보를 다른 문제를 푸는데 사용하는 방식
- Ex)컴퓨터 비전 분야에서 큰 데이터셋을 가지고 이미지 분류 문제를 해결하는데 사용했던 CNN 망을 다른 데이터셋 혹은 다른 문제(task)에 적용시켜 풀.
- 특히 컴퓨터 비전의 영역에서 전이 학습으로 수행된 모델들이 높은 성능을 보이고 있어 가장 많이 사용되는 방법.
- 전이학습을 수행하지 않은 모델들보다 비교적 빠르고 정확한 정확도를 달성 가능

6.1 전이학습의 이해



6.1 전이학습의 이해

그림: Dacon

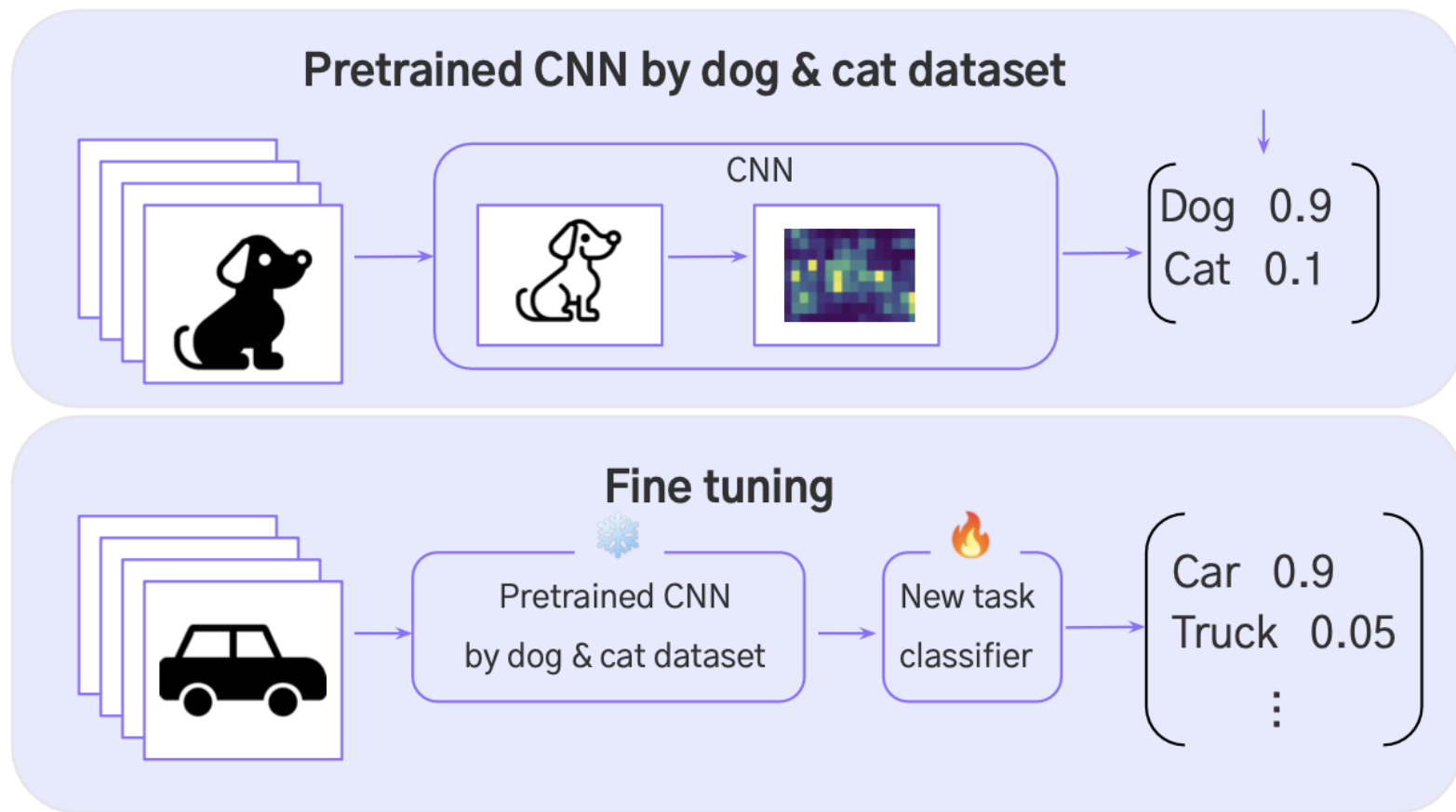


- CNN의 각 컨볼루션 layer에서 학습되는 필터는 데이터가 주어질 때 부분적인 특징을 잡아냄.
- *그러면 이미 잘 학습된 필터들을 약간 변형 혹은 고정시켜서 사용하면 안될까?*

6.1 전이학습의 이해

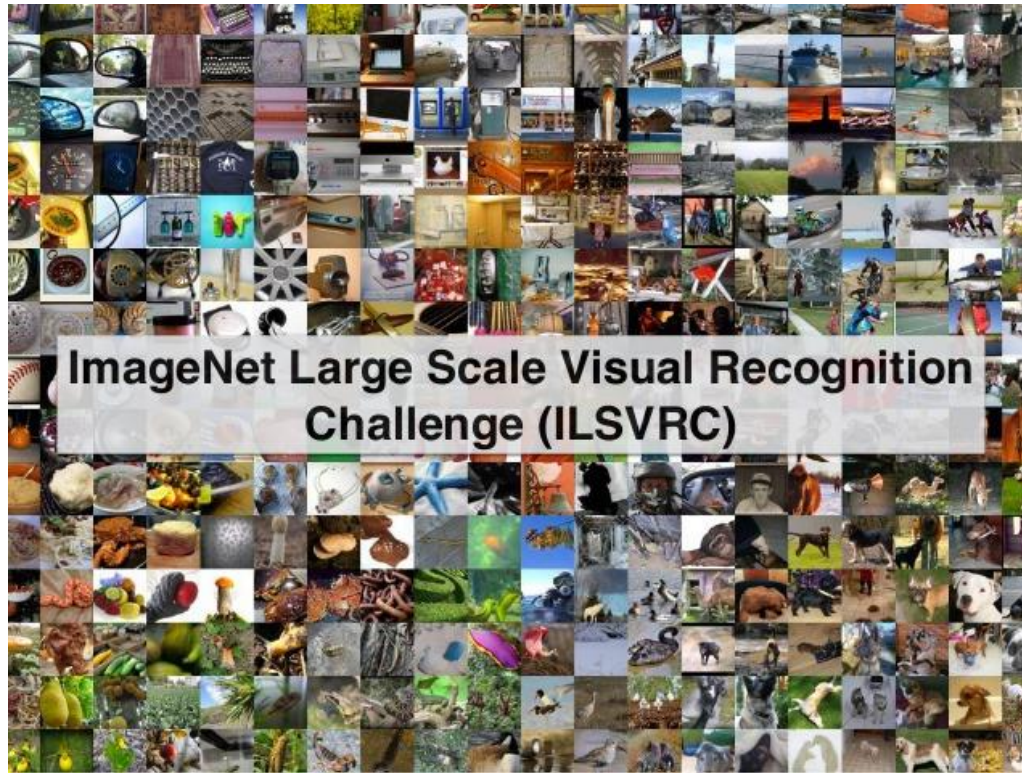
Transfer learning

Fine-tuning



6.1 전이학습의 이해

전이학습의 장점 (데이터 cost 측면)



ImageNet 1k dataset

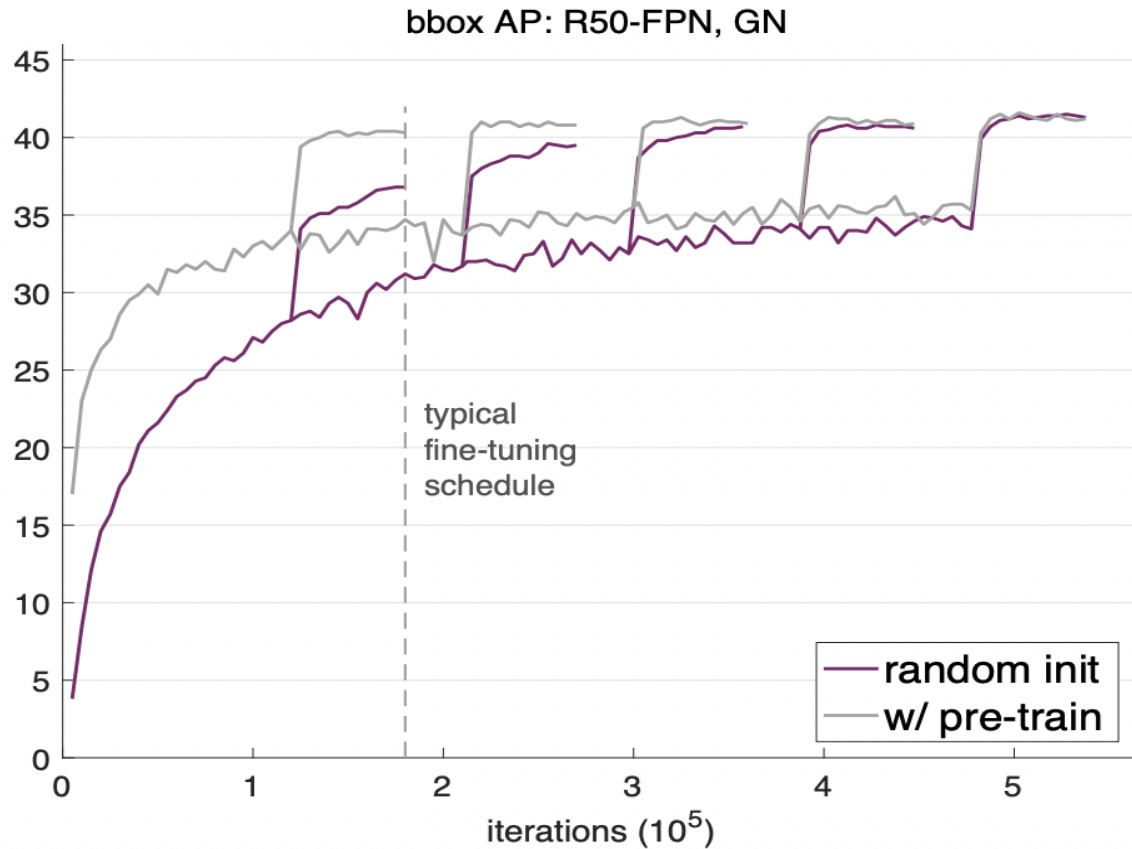
- 대표적인 전이학습용 데이터셋
- 1000개의 카테고리
- 120만개의 학습 데이터 (138GB)
- 5만개의 테스트 데이터 (6GB)

✓ 개인 단위에서 학습 불가...

✓ 하지만 케라스에서 학습 완료된 망을 한 줄로 제공!

6.1 전이학습의 이해

전이학습의 장점 (학습 성능 및 속도 측면)



- 학습 속도 면에서도 의미가 있다
- 같은 성능을 얻는데에 걸리는 학습횟수가 훨씬 적음

Rethinking ImageNet Pre-training (2018, Facebook AI)

6.1 전이학습의 이해

요약

- 전이학습은 큰 데이터에 대해 학습한 지식을 개별 모델의 baseline으로 이용
Ex) CNN에서 이미 학습완료된 신경망 모델의 필터를 가져온다
- Data cost를 줄일 수 있다
- 모델의 성능을 향상 시킬 수 있다
- 모델의 학습 속도를 빠르게 만들 수 있다

전이학습을 이용한 분류 문제 예제

1. 전이학습 기반 cifar-10 이미지 분류 (이미지 데이터 – 컬러)
 - 컬러 이미지 분류모델 설계
2. 전이학습 기반 개/고양이 데이터 이진 분류 (이미지 데이터 – 컬러)
 - 디렉토리부터 라벨링하는 코드
 - 이진 분류에서의 전이학습된 CNN사용

6.2 CNN 전이학습을 이용한 Cifar-10 데이터 분류 모델

데이터셋 설명

- 목적: 10개의 클래스를 가지는 동물사진 정답 예측
- 5만개의 학습 데이터셋
- 1만개의 테스트 데이터셋

Input:

- 28x28 픽셀로 이루어진 컬러 이미지 데이터

Output:

- 최종적인 1개의 물체 class

비행기

자동차

새

고양이

사슴

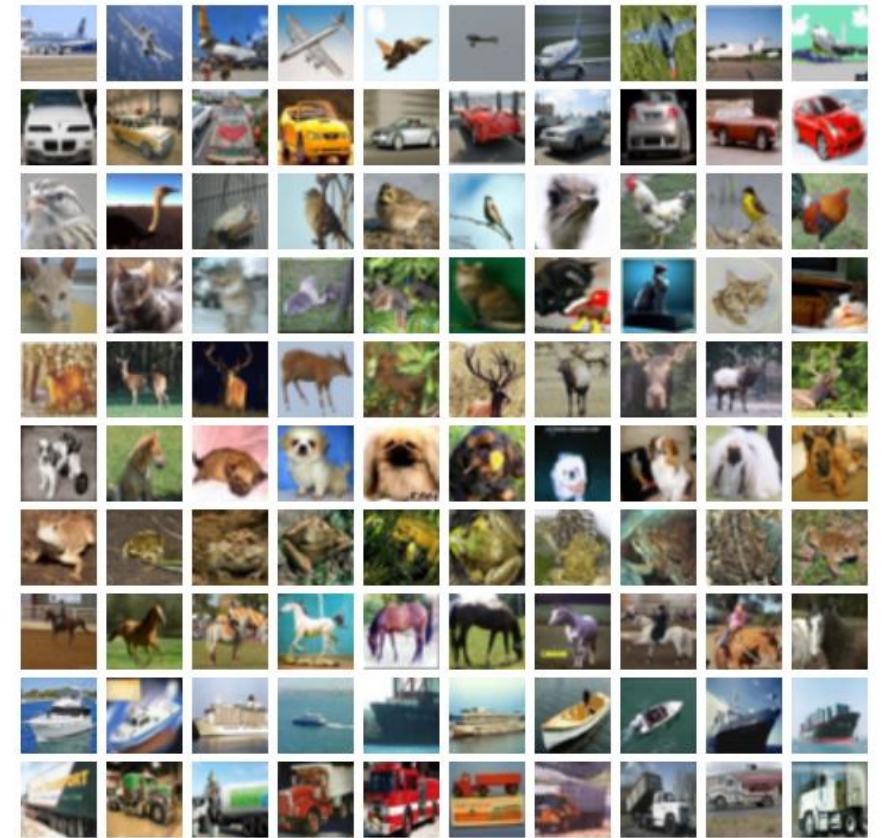
개

개구리

말

배

트럭



Cifar-10 dataset

6.2 CNN 전이학습을 이용한 Cifar-10 데이터 분류 모델

1) Module import

```
import tensorflow as tf
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.applications.vgg16 import VGG16, preprocess_input
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten, Dropout
from tensorflow.keras.optimizers import Adam
import matplotlib.pyplot as plt
```

6.2 CNN 전이학습을 이용한 Cifar-10 데이터 분류 모델

2) Dataset import

```
# CIFAR-10 Dataset 가져오기
(train_images, train_labels), (test_images, test_labels) = cifar10.load_data()
print('x_train :', np.shape(train_images))
print('y_train :', np.shape(train_labels))
print('x_test :', np.shape(test_images))
print('y_test :', np.shape(test_labels))
```

6.2 CNN 전이학습을 이용한 Cifar-10 데이터 분류 모델

3) Visualization

```
# 이미지 예제 시각화
plt.subplot(121)
plt.imshow(train_images[0], interpolation="bicubic")
plt.grid(False)
plt.subplot(122)
plt.imshow(train_images[2], interpolation="bicubic")
plt.grid(False)
plt.show()
```

6.2 CNN 전이학습을 이용한 Cifar-10 데이터 분류 모델

4) Preprocess

이미지 데이터 정규화 및 전처리

```
train_images = preprocess_input(train_images)
```

```
test_images = preprocess_input(test_images)
```

레이블을 원-핫 인코딩

```
train_labels = to_categorical(train_labels, 10)
```

```
test_labels = to_categorical(test_labels, 10)
```


6.2 CNN 전이학습을 이용한 Cifar-10 데이터 분류 모델

5) model

```
# 미리 학습된 VGG16 모델 불러오기 (최상위 레이어는 포함하지 않음)
vgg16 = VGG16(weights='imagenet', include_top=False,
input_shape=(32, 32, 3))

# VGG16 모델의 레이어를 동결 (학습되지 않도록 설정)
for layer in vgg16.layers:
    layer.trainable = False
```

6.2 CNN 전이학습을 이용한 Cifar-10 데이터 분류 모델

5) model

Sequential 모델 구성

```
model = Sequential()  
model.add(vgg16)  
model.add(Flatten())  
model.add(Dense(256, activation='relu'))  
model.add(Dropout(0.5))  
model.add(Dense(10, activation='softmax'))
```

모델 컴파일

```
model.compile(optimizer=Adam(),  
              loss='categorical_crossentropy',  
              metrics=['accuracy'])
```

모델 학습

```
history = model.fit(train_images, train_labels, epochs=10, batch_size=32, validation_data=(test_images,  
test_labels))
```

6.3 CNN 전이학습을 이용한 개/고양이 데이터 분류 모델

1) Module import / parameter setting / 경로설정

```
#압축파일 해제
import zipfile
zipfile.ZipFile('Dataset.zip').extractall()
import os
import random
import warnings
warnings.filterwarnings("ignore")

from keras.applications.vgg16 import VGG16
from keras.models import Model
from keras.layers import Dense, Flatten
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# 초매개변수 정의
INPUT_SIZE = 32
BATCH_SIZE = 15
EPOCHS = 10

#경로 파일
src = 'PetImages/'
```

6.3 CNN 전이학습을 이용한 개/고양이 데이터 분류 모델

1.5) 모델설정

```
from keras.models import Sequential
from keras.layers import Conv2D, MaxPooling2D
from keras.layers import Dropout, Flatten, Dense
from tensorflow.keras.preprocessing.image import ImageDataGenerator

#경로 파일
src = 'PetImages/'

# 초매개변수 정의
FILTER_SIZE = 3
NUM_FILTERS = 32
INPUT_SIZE = 32
MAXPOOL_SIZE = 2
BATCH_SIZE = 16
EPOCHS = 10

model = Sequential()
model.add(Conv2D(NUM_FILTERS, (FILTER_SIZE, FILTER_SIZE), input_shape = (INPUT_SIZE, INPUT_SIZE,
3), activation = 'relu'))
model.add(MaxPooling2D(pool_size = (MAXPOOL_SIZE, MAXPOOL_SIZE)))
model.add(Conv2D(NUM_FILTERS, (FILTER_SIZE, FILTER_SIZE), activation = 'relu'))
model.add(MaxPooling2D(pool_size = (MAXPOOL_SIZE, MAXPOOL_SIZE)))
model.add(Flatten())
model.add(Dense(units = 1, activation = 'sigmoid'))
model.compile(optimizer = 'adam', loss = 'binary_crossentropy', metrics = ['accuracy'])
model.summary()
```

6.3 CNN 전이학습을 이용한 개/고양이 데이터 분류 모델

2) 처음부터 full training 하는 model

```
training_data_generator = ImageDataGenerator(rescale = 1./255)
testing_data_generator = ImageDataGenerator(rescale = 1./255)

training_set = training_data_generator.flow_from_directory(src+'Train/',
                                                         target_size = (INPUT_SIZE, INPUT_SIZE),
                                                         batch_size = BATCH_SIZE,
                                                         class_mode = 'binary')

test_set = testing_data_generator.flow_from_directory(src+'Test/',
                                                     target_size = (INPUT_SIZE, INPUT_SIZE),
                                                     batch_size = BATCH_SIZE,
                                                     class_mode = 'binary')

print(training_set)

model.fit(training_set, epochs = EPOCHS, verbose=1)

score = model.evaluate(test_set)

for idx, metric in enumerate(model.metrics_names):
    print("{}: {}".format(metric, score[idx]))
```

- flow_from_directory ??

6.3 CNN 전이학습을 이용한 개/고양이 데이터 분류 모델

2.5) VGG16 network를 그대로 가져와서 fine-tuning하는 model

```
vgg16 = VGG16(include_top=False, weights='imagenet', input_shape=(INPUT_SIZE, INPUT_SIZE, 3))

# 사전 학습된 레이어 고정
for layer in vgg16.layers:
    layer.trainable = False

# 신경망 맨 오른쪽에 노드 1개짜리 완전 연결 레이어 추가
print(vgg16.input)
input_ = vgg16.input
output_ = vgg16(input_)
last_layer = Flatten(name='flatten')(output_)
last_layer = Dense(1, activation='sigmoid')(last_layer)
model = Model(inputs=input_, outputs=last_layer)
model.compile(optimizer = 'adam', loss = 'binary_crossentropy', metrics = ['accuracy'])
model.summary()
```

- Model이라는 함수는 input과 output 설정 가능

6.3 CNN 전이학습을 이용한 개/고양이 데이터 분류 모델

3) 결과 확인

```
training_data_generator = ImageDataGenerator(rescale = 1./255)
testing_data_generator = ImageDataGenerator(rescale = 1./255)

training_set = training_data_generator.flow_from_directory(src+'Train/',
                                                           target_size = (INPUT_SIZE, INPUT_SIZE),
                                                           batch_size = BATCH_SIZE,
                                                           class_mode = 'binary')

test_set = testing_data_generator.flow_from_directory(src+'Test/',
                                                      target_size = (INPUT_SIZE, INPUT_SIZE),
                                                      batch_size = BATCH_SIZE,
                                                      class_mode = 'binary')

hist=model.fit(training_set,test_set, epochs = EPOCHS, verbose=1)
score = model.evaluate(test_set)

for idx, metric in enumerate(model.metrics_names):
    print("{}: {}".format(metric, score[idx]))
```

6.3 CNN 전이학습을 이용한 개/고양이 데이터 분류 모델

4) 결과 시각화

```
plt.figure(1, figsize = (15,8))

plt.subplot(221)
plt.plot(hist.history['accuracy'])
plt.plot(hist.history['val_accuracy'])
plt.title('model accuracy')
plt.ylabel('accuracy')
plt.xlabel('epoch')
plt.legend(['train', 'valid'])

plt.subplot(222)
plt.plot(hist.history['loss'])
plt.plot(hist.history['val_loss'])
plt.title('model loss')
plt.ylabel('loss')
plt.xlabel('epoch')
plt.legend(['train', 'valid'])

plt.show()
```


6.3 CNN 전이학습을 이용한 개/고양이 데이터 분류 – version2

```
#경로 파일
src = 'PetImages/'
vgg16 = VGG16(include_top=False, weights='imagenet', input_shape=(INPUT_SIZE, INPUT_SIZE, 3))

# 사전 학습된 레이어 고정
for layer in vgg16.layers[:-3]:
    layer.trainable = False

# 신경망 맨 오른쪽에 노드 1개짜리 완전 연결 레이어 추가
print(vgg16.input)
input_ = vgg16.input
output_ = vgg16(input_)
last_layer = Flatten(name='flatten')(output_)
last_layer = Dense(1, activation='sigmoid')(last_layer)
model = Model(inputs=input_, outputs=last_layer)
model.compile(optimizer = 'adam', loss = 'binary_crossentropy', metrics = ['accuracy'])
model.summary()

training_data_generator = ImageDataGenerator(rescale = 1./255)
testing_data_generator = ImageDataGenerator(rescale = 1./255)

training_set = training_data_generator.flow_from_directory(src+'Train/',
                                                         target_size = (INPUT_SIZE, INPUT_SIZE),
                                                         batch_size = BATCH_SIZE,
                                                         class_mode = 'binary')

test_set = testing_data_generator.flow_from_directory(src+'Test/',
                                                     target_size = (INPUT_SIZE, INPUT_SIZE),
                                                     batch_size = BATCH_SIZE,
                                                     class_mode = 'binary')

model.fit(training_set, epochs = EPOCHS, verbose=1)
score = model.evaluate(test_set)

for idx, metric in enumerate(model.metrics_names):
    print("{}: {}".format(metric, score[idx]))
```

CNN의 마지막 3개 layer를 학습
/ 나머지 layer는 freezing

6.3 CNN 전이학습을 이용한 개/고양이 데이터 분류 – version3

```
from keras.applications.resnet import ResNet50
src = 'PetImages/'
resnet50 = ResNet50(include_top=False, weights='imagenet', input_shape=(INPUT_SIZE, INPUT_SIZE, 3))

# 사전 학습된 레이어 고정
for layer in resnet50.layers[:-3]:
    layer.trainable = False

# 신경망 맨 오른쪽에 노드 1개짜리 완전 연결 레이어 추가
print(resnet50.input)
input_ = resnet50.input
output_ = resnet50(input_)
last_layer = Flatten(name='flatten')(output_)
last_layer = Dense(1, activation='sigmoid')(last_layer)
model = Model(inputs=input_, outputs=last_layer)
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
model.summary()

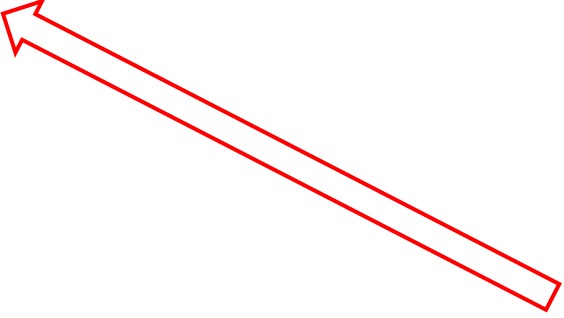
training_data_generator = ImageDataGenerator(rescale=1./255)
testing_data_generator = ImageDataGenerator(rescale=1./255)

training_set = training_data_generator.flow_from_directory(src+'Train/',
                                                         target_size=(INPUT_SIZE, INPUT_SIZE),
                                                         batch_size=BATCH_SIZE,
                                                         class_mode='binary')

test_set = testing_data_generator.flow_from_directory(src+'Test/',
                                                      target_size=(INPUT_SIZE, INPUT_SIZE),
                                                      batch_size=BATCH_SIZE,
                                                      class_mode='binary')

model.fit(training_set, epochs=EPOCHS, verbose=1)
score = model.evaluate(test_set)

for idx, metric in enumerate(model.metrics_names):
    print("{}: {}".format(metric, score[idx]))
```

- 
- VGG16 => Resnet 으로 변경

감사합니다